

Django Student Management System – Ultra Detailed Explanation (Complete Guide)

This document is a **full beginner-to-intermediate explanation** of your entire Django project. It includes:

- Definitions of every concept
 - What you did step-by-step
 - How each feature was implemented
 - All major functions used
 - Database (SQL) explanation and connection
-

0. What You Built (Overview)

You built a **Student Management System** using Django.

It allows:

- User Authentication (login/logout)
 - Student CRUD (Create, Read, Update, Delete)
 - Marks management
 - Percentage calculation
 - Dashboard analytics
-

1. Django (Definition)

Django is a **high-level Python web framework** used to build web applications quickly.

Architecture: MVT

- Model → Database structure
 - View → Business logic
 - Template → Frontend/UI
-

2. Project Setup (Implementation)

Commands used:

```
django-admin startproject sms_project  
cd sms_project  
python manage.py startapp students
```

Explanation:

- `startproject` → creates main project
 - `startapp` → creates module (students)
-

3. App Registration

File: settings.py

```
INSTALLED_APPS = ['students']
```

Definition:

INSTALLED_APPS = list of apps Django should load

Why:

Without this → Django will ignore your app

4. Models (Database Layer)

Definition:

Model = Python class representing a database table

Student Model

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    roll = models.IntegerField()
    age = models.IntegerField()
    course = models.CharField(max_length=100)
    user = models.ForeignKey(User, on_delete=models.CASCADE)
```

Explanation line-by-line:

- `models.Model` → tells Django this is a DB table
 - `CharField` → text column
 - `IntegerField` → number column
 - `ForeignKey` → relationship (link to another table)
-

Marks Model

```
class Marks(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    subject = models.CharField(max_length=100)
    marks = models.IntegerField()
```

Concept:

One student → many marks (One-to-Many relationship)

5. Migrations (Database Creation)

Commands:

```
python manage.py makemigrations
python manage.py migrate
```

Definition:

Migration = converting model changes into SQL queries

What happens:

- Django generates SQL
 - Creates tables in SQLite
-

6. Database (SQL Explanation)

Database Used:

SQLite (default Django database)

File:

```
db.sqlite3
```

Example SQL Generated:

```
CREATE TABLE students_student (  
    id INTEGER PRIMARY KEY,  
    name VARCHAR(100),  
    roll INTEGER,  
    age INTEGER,  
    course VARCHAR(100),  
    user_id INTEGER  
);
```

How Django Connects to DB

settings.py:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

Explanation:

- ENGINE → type of database
 - NAME → database file
-

7. Authentication System

Functions used:

```
authenticate()  
login()  
logout()
```

Definitions:

- authenticate → checks credentials
 - login → starts session
 - logout → ends session
-

8. Views (Logic Layer)

Definition:

View = Python function handling request & response

Example:

```
def student_list(request):
```

Explanation:

- request → contains user/browser data
 - function returns HTML response
-

9. CRUD Operations

Definition:

CRUD = Create, Read, Update, Delete

Create (Add Student)

```
form = StudentForm(request.POST)
```

- request.POST → user input data
-

```
form.is_valid()
```

- checks correctness
-

```
form.save()
```

- inserts into DB
-

10. Templates (Frontend)

Definition:

Templates = HTML with dynamic data

Example:

```
{{ student.name }}
```

Meaning:

Display backend variable

11. Bootstrap

Definition:

CSS framework for styling

Used for:

- Buttons
- Tables
- Layout

12. Search System

```
Q(name__icontains=query)
```

Definition:

Q = complex query builder

Meaning:

Search text inside field

13. Pagination

```
Paginator(students, 5)
```

Definition:

Pagination = splitting data into pages

14. Marks System

```
Marks.objects.filter(student=student)
```

Definition:

QuerySet = collection of DB objects

15. Percentage Calculation

```
total = sum(m.marks for m in marks)
```

Definition:

Aggregation = combining values

```
percentage = total / count
```

16. Dashboard

Implemented using:

```
students.count()  
marks.aggregate(Avg('marks'))
```

Definition:

Aggregate = DB-level calculation

17. Top Performer Logic

```
for student in students:
```


Definition:

Loop = iterate through data

18. URL Routing

```
path('add/', views.add_student)
```

Definition:

URL dispatcher = connects URL to view

19. Messages Framework

```
messages.success(request, "...")
```

Definition:

Temporary notifications system

20. Functions Used (Complete List)

Django Functions:

- render()
- redirect()
- get_object_or_404()
- authenticate()
- login()
- logout()

ORM Functions:

- objects.filter()
- objects.get()
- objects.create()
- objects.count()
- aggregate()

Python Functions:

- `sum()`
 - `round()`
-

21. Full Flow of Application

User → Request → URL → View → Model → Database → Template → Response

FINAL SUMMARY

You built a **full-stack Django application** with:

- Authentication
- CRUD system
- Relational database
- Analytics dashboard

This is a **strong intermediate-level project** suitable for internships.